

Word Representation

Introduction to NLP — MSc. DH ENC-PSL

Noé Durandard (noe.durandard@psl.eu)

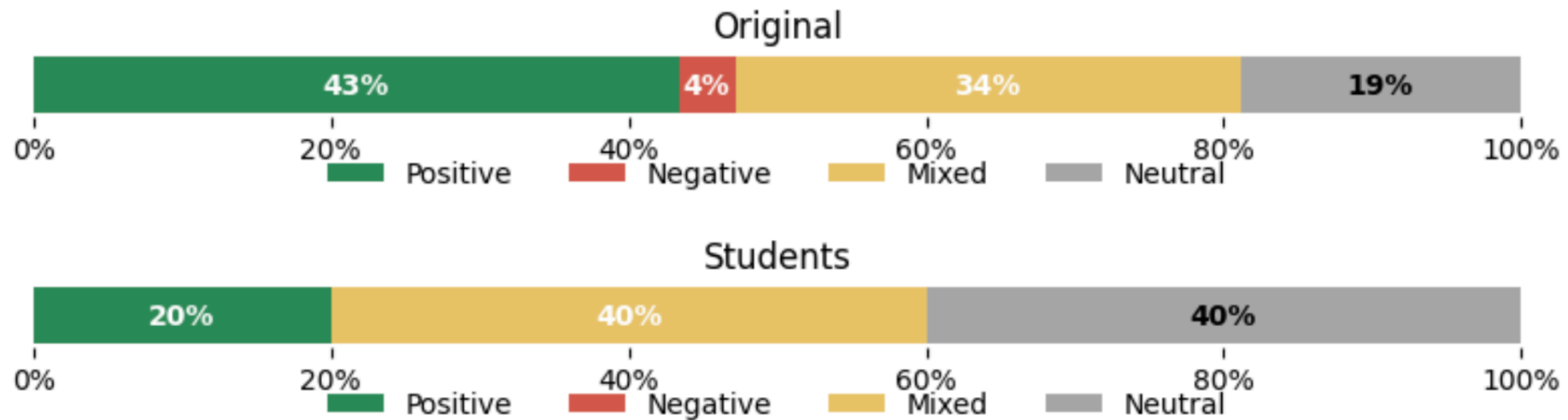
October 7, 2026

1 Recap

- NLP deals with **natural** (ambiguous, fluid, evolving, ...) **language**
- The field has evolved toward **learning from data**...
- ... which is not inherently objective but results from a *construction of truth*.

[Supplications_and_Requests] (Ground Truth: Negative)

Let us not forget how hard we had to work to get to where we are now



Sample Analysis from last week's *Annotation Workshop*.

Table of Contents

- 1 Recap
- 2 Tokenization & Representation
- 3 The meaning of meaning
- 4 Geometry aside
- 5 Dense Representations: Word2Vec
- 6 Navigating Embedding Spaces
- 7 Conclusion

2 Tokenization & Representation

2.1 Motivation

2.1.1 The symbolic gap

- Computers do **not** process words, or concepts directly; they compute on **numbers, matrices, and vectors**.
- **Raw Text in Memory:** A continuous stream of Unicode bytes or ASCII characters.

2.1.2 Definition

Tokenization

the task of separating out or tokenizing words and word parts from running text.
([Jurafsky and Martin 2026](#))

Process flow:

Raw text → **tokenize** → Token sequence → **vocabulary**
lookup → Vocabulary IDs

2.1.3 Is it **token** = **word**?

→ What is a *word*?

- **cat** (standard, unambiguous)
- **I 'm** (one orthographically, two grammatically)
- **San Francisco** (two orthographically, one semantic entity)
- **C U** (abbreviation / chat speak)
- What about languages without whitespace separators?

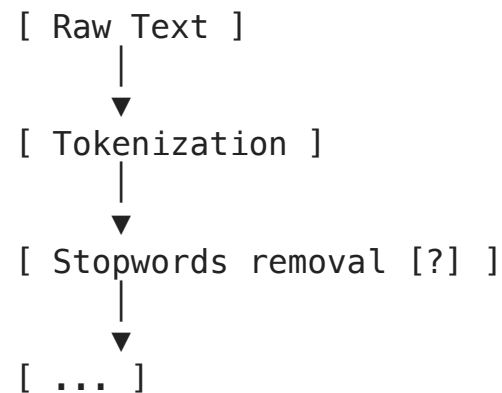
The concept of a “word” is *ill-defined*, *context-bound*, and *language-dependent*.

Note: For simplicity, you may think of **token** $\approx$ **word**, but keep in mind that these notions differ in practice.

2.1.4 The universal first step

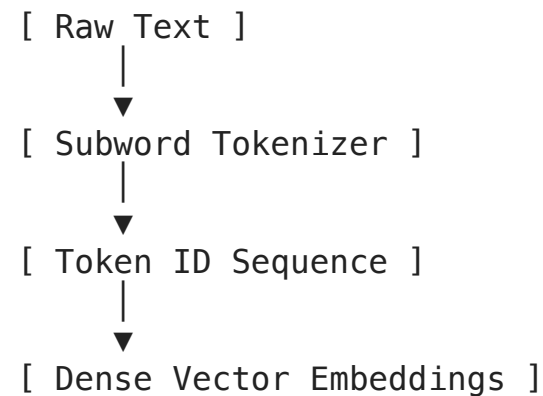
Regardless of historical era or model paradigm, **tokenization is the gateway to every NLP pipeline:**

Traditional NLP Pipeline



- Discrete rule-based or statistical features.

Modern Neural Pipeline



- Continuous vector spaces & Transformers.

2.2 Tokenization Approaches

2.2.1 Tokenizer's Desiderata

Raw text → **tokenizer** → discrete **tokens** from **vocabulary** □.

That vocabulary and mapping should be:

- **Expressive**

- able to represent any given text (any language)
- avoid missing elements and cleanly represent variants

- **Efficient**

- not too large: large vocabulary ⇒ bloated lookup tables, more parameters to store (/learn).
- not too small: smaller vocabulary ⇒ longer sequences, computationally expensive.

2.2.2 ‘Word’-Level & Pattern matching

Using regular expressions to isolate ‘words’ explicitly:

```
1 import re
2 text = "The cat is on the mat, the dog isn't."
3 tokens = re.findall(r"\w+|[\^\\w\\s]", text)
4 # Output: ['The', 'cat', 'is', 'on', 'the', 'mat', ',', 'the', 'dog', 'isn']
```

Limitations: **Size Explosion, Out-Of-Vocabulary (OOV)** issues, rely on **whitespace delimiters** (what about other languages: Chinese, Japanese, Ancient Greek *scriptio continua*, etc.?)



Regex as pre-tokenization

Note that such approaches are still the foundation *pre*-tokenization step of modern tokenizers!

It is used to prevent from merging across punctuation and space boundaries (more shortly).

2.2.3 Character- & Byte-Level

Split text into raw individual characters or UTF-8 bytes:

```
1 text = "the cat"
2 chars = list(text)
3 utf8s = list(text.encode('utf-8'))
4 # chars: ['t', 'h', 'e', ' ', 'c', 'a', 't']
5 # utf8s : [116, 104, 101, 32, 99, 97, 116]
```

Pros:

- Complete expressivity (any unicode string)
- Very small vocabulary: UTF-8 = 256 bytes

Cons:

- Inefficient: yield very long sequences
- Weak individual semantics

2.2.4 Subword-level

Goal: Find middle ground between **word-level** (rich semantics, high OOV risk, large vocabulary) and **character-level** (zero OOV, tiny vocabulary, long sequence length, poor semantics) tokenizers.

How?

→ **Data-Driven & Statistical:** Learn optimal (subword) decomposition from corpus statistics.

2.2.4.1 Byte-Pair Encoding (BPE) ([Gage 1994](#))

Key Idea

Start from a **base vocabulary** (e.g., UTF-8) and **iteratively merge** the most common **token pairs** in the corpus into new tokens.

Initially for data compression ([Gage 1994](#)), adapted by Sennrich, Haddow, and Birch ([2016](#)) for neural NLP.

```
1 function BYTE-PAIR ENCODINGS(C:string, vocab_size:int):
2     V = [all unique bytes in C] # initial set of tokens is characters (bytes)
3
4     learned_merges = [] # list of learned merges
5     while len(V) < vocab_size: # expand vocab (merges) until reaching vocab_s
6         (a, b) = get_most_frequent_pair(C) # most frequent adjacent pair
7         new_token = a + b # make new token by concatenation
8         V += [new_token]
9         learned_merges += [(a,b), new_token]
10        update(C, learned_merges) # update corpus: replace each occurrence of (a
11
12    return V
```

2.2.5 Some useful libraries

Different libraries specialize in different tokenization paradigms:

Library	Paradigm	Typical Level	Primary Use Case
NLTK	Rule-based & regex heuristics	Words / Sentences	classical NLP prototyping
spaCy	Dependency-aware tokenization	Words / Morphemes	Production linguistic pipelines (POS, NER)
tiktoken	Byte-Pair Encoding (BPE)	Subwords / Bytes (usually ~ whitespace pre-tokenization)	OpenAI LLM models (GPT-3.5, GPT-4o)
sentencepiece	~ Byte-Pair Encoding (BPE)	Subwords / Bytes (raw stream)	LLMs (e.g., Llama)

2.3 Tutorial

2.4 From tokens to vectors

2.4.1 Vocabulary mapping & One-Hot-Encoder

Every single token in $\square$ is assigned an integer index:

$$\square = \{ \text{"cat"} : 0, \text{"mat"} : 1, \text{"on"} : 2, \text{"sat"} : 3, \text{"the"} : 4, \}$$

To be processed by mathematical models, they are casted into **vectors**.

The most straightforward approach is to use **one-hot-encoder**: $\mathbf{x} \in \{0, 1\}^{|\square|}$, with $x_i = 1$ for a *unique* value of i , its ID in the vocabulary.

$$\mathbf{x}_{\text{sat}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

2.4.2 OHE Limitations

- Vocabulary can get very large (easily $|\Sigma| > 10^5$), so $\mathbf{x}$ lives in very large dimensions $\mathbb{R}^d$
- Potentially huge footprint (though there are ways to handle sparse vectors) & ‘curse of dimensionality’
- All distinct words are mathematically orthogonal! There is no notion of similarity or semantic.

3 The meaning of *meaning*

3.1 Distributional Semantics

“You shall know a word by the company it keeps.”

J.R. Firth (1957)

3.1.1 What's the _____ word?

- Recently, my friend _____ 20 packets of kolo mee before she flew back to Kuala Lumpur.

-  tapao ([Wiktionary](#))

Verb (mostly invariable, ambitransitive; Malaysia, Singapore, Manglish, Singlish):

To order (food) as takeaway; to pack food so it can be consumed elsewhere.

Etymology: Borrowed from Cantonese 打包 (*daa³ baau¹*, “to pack”).

Pronunciation: [t̚ʰɑ(ː)ɿp̚ʰaʊ(ɿ)]

nearest restaurant to (physically) _____ our food... in the rain... while it was all jammed up.

3.1.2 The distributional hypothesis

Meaning = use

- “[T]he meaning of a word is its use in the language.” ([Wittgenstein 1953](#))

Meaning = environment

- “You shall know a word by the company it keeps.” ([Firth 1957](#))
- “If A and B have almost identical environments, we say that they are synonyms. [...] In other words, difference of meaning correlates with difference of distribution.” ([Harris 1954](#))



Core Takeaway

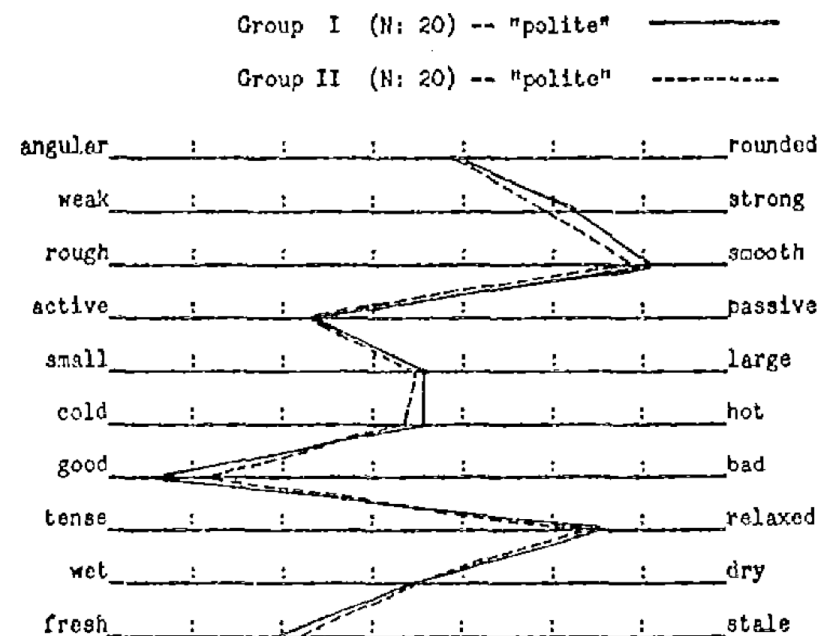
Similar use / environment $\Rightarrow$ similar aspects of meaning. Context becomes our proxy for semantics.

3.2 Semantic Spaces

3.2.1 Mapping words into space

Meaning = point in space

- Can we formalize word meaning as a point in a *multi-dimensional geometric space*?
- Pioneered in psycholinguistics with Osgood (1952)'s *Semantic Differentials*
 - Participants rate concepts along bipolar adjective scales
 - Evaluation-Potency-Activity (EPA) dimensions



Semantic differentials, averaged across two groups of twenty study participants each. *Extracted from (Osgood 1952)*

3.2.2 Computational desiderata

Computers require numerical representations—but **how** should those numbers be structured?

! Symbolic / One-Hot Space

- Words as arbitrary discrete IDs.
- **Purely structural:** Every word is perpendicular (orthogonal) to every other word.
- $\mathbf{v}_{\text{cat}} \cdot \mathbf{v}_{\text{dog}} = 0$
- Distance conveys **no semantic information**.

i Continuous Semantic Space

- Words as coordinates in a shared space.
- **Semantically structured:** Spatial relationships encode linguistic properties.
- Geometric distance $\approx$ Semantic difference.

The Goal: Build a vector space where **geometry reflects semantics**.

3.3 Count-based semantic vectors

3.3.1 Key idea

Word-word co-occurrence matrix: Use the *Distributional Hypothesis* to embed words in a *Semantic Space*.

1. Define a **vocabulary** $\square$ and a target word w .
2. Define a **context window** (e.g., ± 2 or ± 5 words around w).
3. Construct a **Term-Context Co-occurrence Matrix**

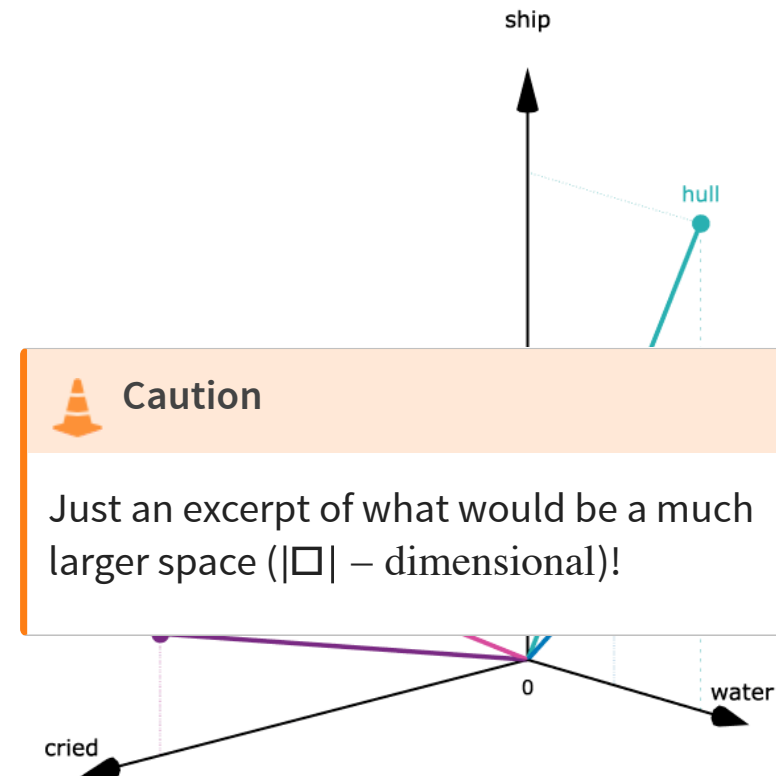
$$\mathbf{M} \in \mathbb{R}^{|\square| \times |\square|}:$$

- M_{ij} = count of word j appearing in the context window of word i .

→ Each row of the matrix becomes a **distributional vector** representing its associate word in $|\square|$ -dimensional space!

3.3.2 Example

- your eyes, **bildad**,” cried **peleg**, “thou
- ’st splice hands,” cried **peleg**, drawing nearer
- , hear him now,” cried **peleg** ,marching across
- the other part of the **ship**, captain **peleg** ripped
- “**bildad**,” cried captain **peleg**, ”
- “first congregational church,” cried **bildad**, “what
- do tell, now,” cried **bildad**, “is
- captain, and finally a **ship** owner; **bildad**,
- **bildad**, and get the **ship**’s papers.
- **keel**; but turning under **water**, swiftly shot to
- my conscience is in this **ship**’s **keel**.–
- her great **hull** through the **water**, as a horse
- whale lay rolling in the **water** like the capsized **hull**
- as the mark in a **ship**’s **hull**,
- in the pitchiest night the **ship**’s black **hull**
- the **hull** of a great **ship** new - laid upon
- nearing black **hull** of the **ship**; seemingly seeing in



Vectors constructed from Moby Dick (Melville, 1851), context window size = 5.

3.3.3 Other methods?

- Count models

- Matrix models

- word-word co-occurrence
 - Latent Semantic Analysis (LSA)
 - GloVe

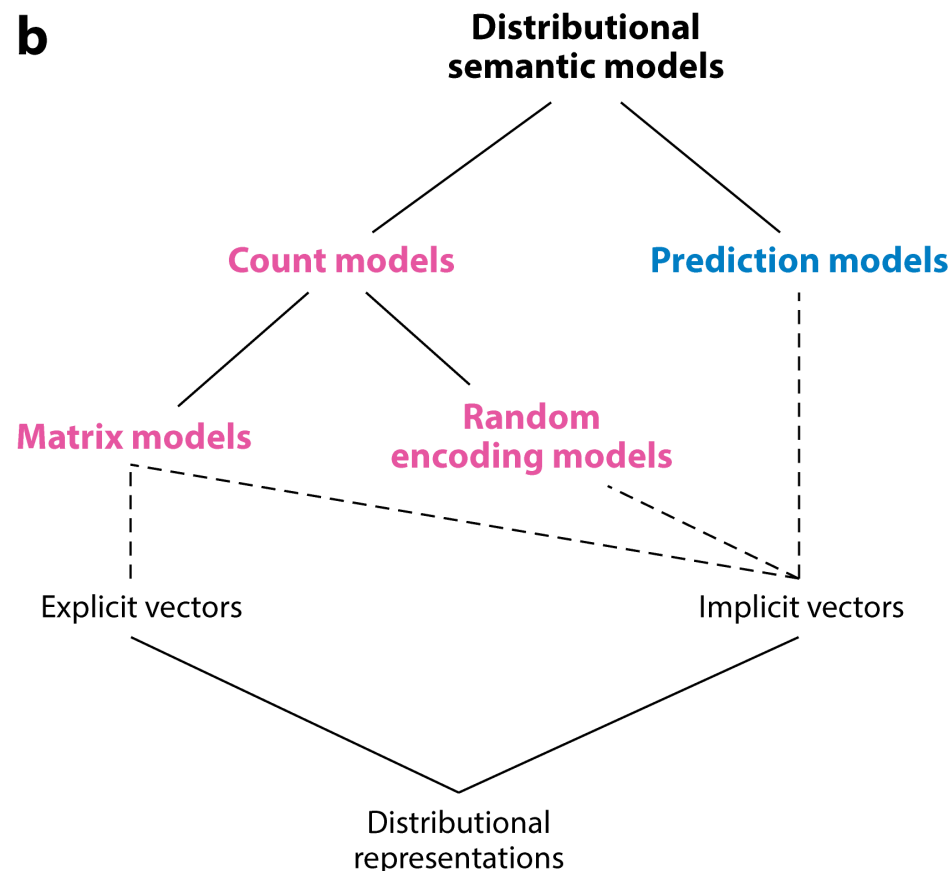
- Random encoding models

- Random indexing (RI)

- Prediction models

- word2vec

- fasttext



Taxonomy of distributional semantic models based on methods to build distributional vectors. *Extracted from (Lenci 2018).*

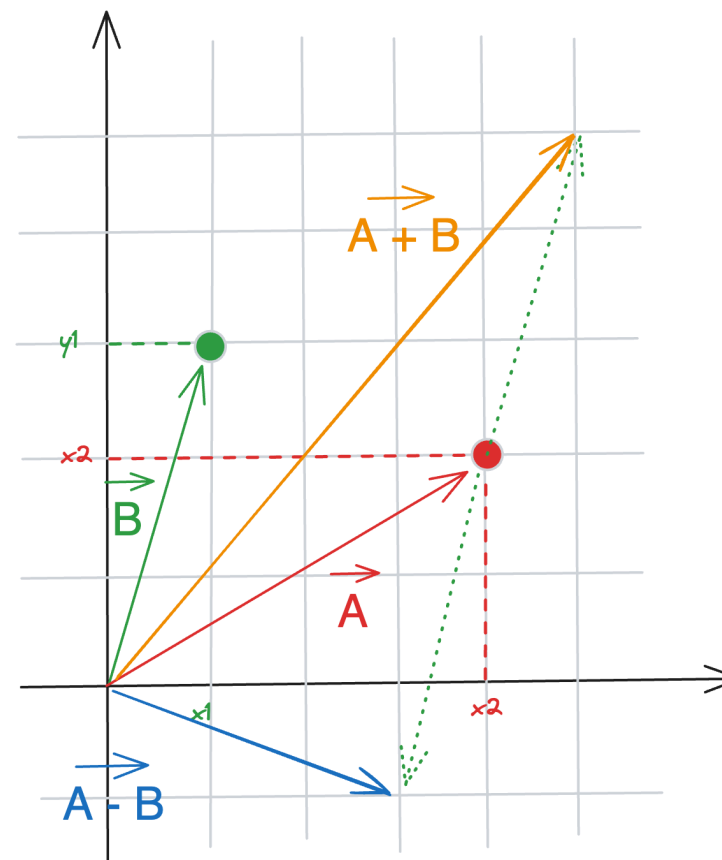


4 Geometry aside

4.1 Vector Arithmetic

4.1.1 Basics

- $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$
- $a_i \in \mathbb{R} \ \forall i \in \{1, \dots, d\}$
- $\mathbf{a} = \sum_{i=1}^d a_i \hat{\mathbf{e}}_i$
- Sum: $\mathbf{a} + \mathbf{b} = \sum_{i=1}^d (a_i + b_i) \hat{\mathbf{e}}_i$
- Inner dot product:
 $\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \sum_{i=1}^d a_i b_i$



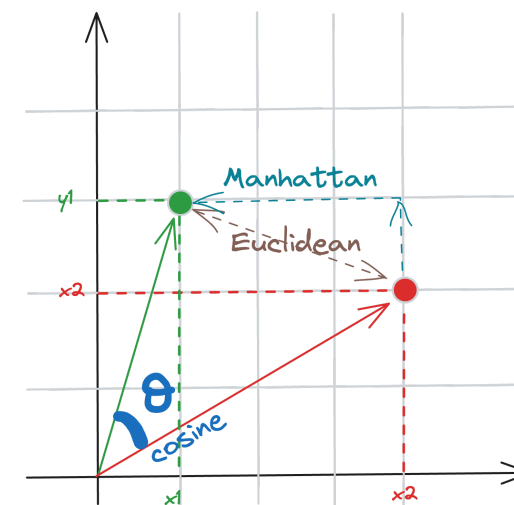
2D diagram

4.2 Notions of distances

4.2.1 Geometrical Distances in 2 dimensions

Different notions of distance, e.g.:

- **Euclidean:** $\sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$
- **Manhattan (L1):** $|x_1 - y_1| + |x_2 - y_2|$
- **Cosine:** $1 - \cos \theta = 1 - \frac{x_1 y_1 + x_2 y_2}{\sqrt{x_1^2 + x_2^2} \sqrt{y_1^2 + y_2^2}}$
 - ⚠ formally cosine distance is not a metric, but a dissimilarity measure



2D diagram.

4.2.2 Geometrical Distances in **N** dimensions

Different notions of distance, e.g.:

- **Euclidean:** $d(x, y) = \sqrt{\sum_{i=1}^N (x_i - y_i)^2}$
- **Manhattan (L1):** $d(x, y) = \sum_{i=1}^N |x_i - y_i|$
- **Cosine:** $1 - \cos \theta = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$
 - ⚠️ formally not a metric, but a dissimilarity measure

Mathematical Formalization

1. The distance between an object and itself is always zero. 
2. The distance between distinct objects is always positive.
3. Distance is symmetric.
4. Distance satisfies the triangle inequality.

4.2.3 Beware of the curse!

⚠ Curse of dimensionality (*coined by* ([Bellman 1966](#))) (see e.g.: ([Domingos 2012](#)))

- **Sparsity:** data points occupy a tiny fraction of the space.
- **Distance collapse:** distances between points converge → “everything is equally far.”
- **Implication:** similarity measures lose discriminative power.

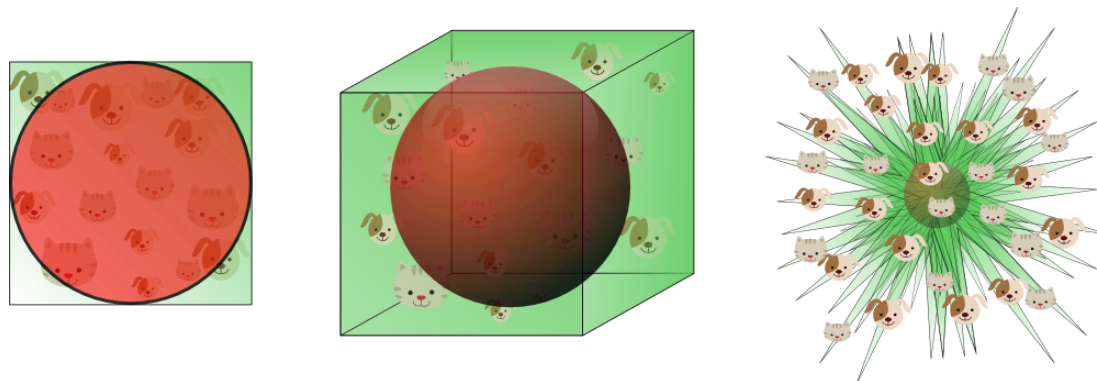


Illustration of the curse of dimensionality: As dimensionality increases, a larger percentage of the data resides at the border of the feature space. Extracted from ([Spruyt 2015](#)).

4.2.4 In practice: (most often) cosine similarity

Dot Product:

$$\mathbf{A} \cdot \mathbf{B} = \|\mathbf{A}\| \|\mathbf{B}\| \cos \theta$$

Cosine similarity:

$$\cos \theta = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$



Origins

Use of cosine similarity for related tasks is inherited from Information Retrieval and Vector Space Models ([Salton, Wong, and Yang 1975](#)).



No Free Lunch

Mathematically also suffers from the curse of dimensionality, but **empirically** works better in practice (most often)...

... still has known limitations, e.g.:

- can sometimes yield arbitrary (and therefore meaningless) ‘similarities’ ([Steck, Ekanadham, and Kallus 2024](#));
- relative to human judgements, cosine underestimates similarity of frequent words ([Zhou et al. 2022](#)).

4.3 Dimensionality Reduction

4.3.1 From high- to lower-dimensional spaces

Can serve different purposes:

- Mitigate curse of dimensionality
- Visualisation & Exploration
- Analytical

→ Reminder (?): various methods exist (MDS, PCA, tSNE, UMAP, ...)

4.3.2 Quick Cheatsheet

MDS (Multidimensional Scaling)

- Preserves pairwise **distances**.
- Good for small datasets, metric interpretability.
- Computationally expensive (to untractable) for large datasets.

Key params: `metric`, `n_init`, `max_iter`

PCA (Principal Component Analysis)

- Linear projection maximizing **variance**.
- Fast, deterministic, interpretable axes.
- Captures global but not nonlinear structure.

t-SNE (t-Distributed Stochastic Neighbor Embedding)

- Preserves **local neighborhoods**.
- Excellent for cluster visualization.
- Nonlinear, stochastic, slower.

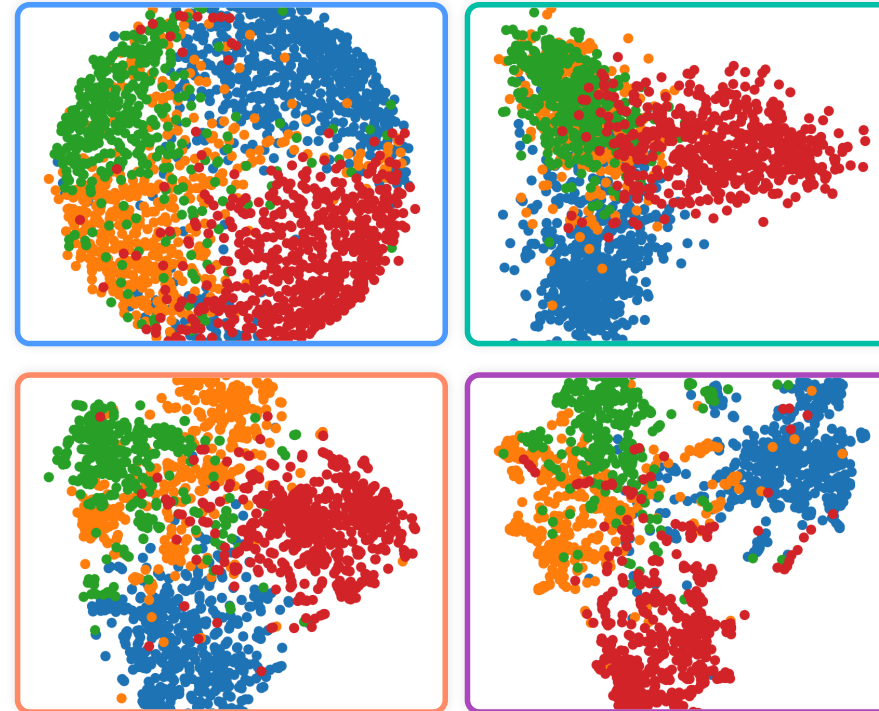
Key params: `perplexity`, `learning_rate`, `n_iter`

UMAP (Uniform Manifold Approximation and Projection)

- Preserves **local + global** structure.
- Scalable and fast.
- Good for semantic visualization of embeddings.

Key params: `n_neighbors`, `min_dist`, `metric`

💡 **Tip:** Use *PCA* for preprocessing and transformation interpretability, *t-SNE* or *UMAP* for visualization, *MDS* for preserving metric geometry.



🔬 **Experiment:** Illustrations based on ~2K excerpts of 20Newsgroups, covering 4 topics. Dimensionality reduction methods applied to 384-dimensional embeddings obtained with `all-MiniLM-L6-v2` SentenceTransformer, using cosine distance for MDS and default arguments from `sklearn` and `umap` libraries for PCA, t-SNE, and UMAP.

5 Dense Representations: Word2Vec

5.1 Paradigm Shift: Count vs. Predict

5.1.1 Why Move Beyond Count Matrices?

Recall our count-based $M \in \mathbb{R}^{|\square| \times |\square|}$ co-occurrence matrix:

→ Representations are **long** ($\sim 10^5$) and **sparse** (mostly zeros),
we would like **short** ($\sim 10^2$) and **dense** (mostly non zeros):

- more efficient computationally
- easier to use as features in ML (fewer weights to tune)
- may generalize better than explicit count
- may represent meaning better
- **simply work better in practice**

5.1.2 Prediction as Representation

Instead of **counting frequencies**, we play a continuous **guessing game**: *Is word w_A likely to show near w_B ?*

- **Train a classifier on a prediction task**
 - **self-supervised** training on a corpus, “gold labels” directly from the corpus
 - pretext task: we are not really interested in the actual predictions
- The (semantic vectors =) **embeddings** are the **learned weights**

Goal

Make each vector “know” about the contexts/environments of its associated word.

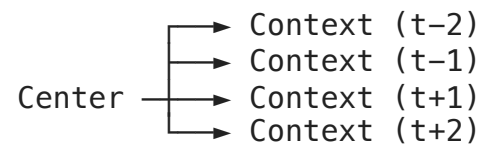
5.2 Word2Vec Algorithms

5.2.1 Two Complementary Approaches

Mikolov, Chen, et al. (2013) introduced two distinct model topologies:

1- Skip-gram

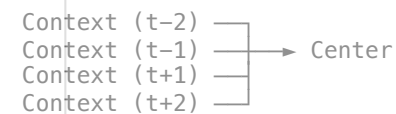
Given a **Center Word** (c), predict its surrounding **Context Words** (o).



- **Strengths:** Works exceptionally well on smaller corpora; captures rare words effectively.

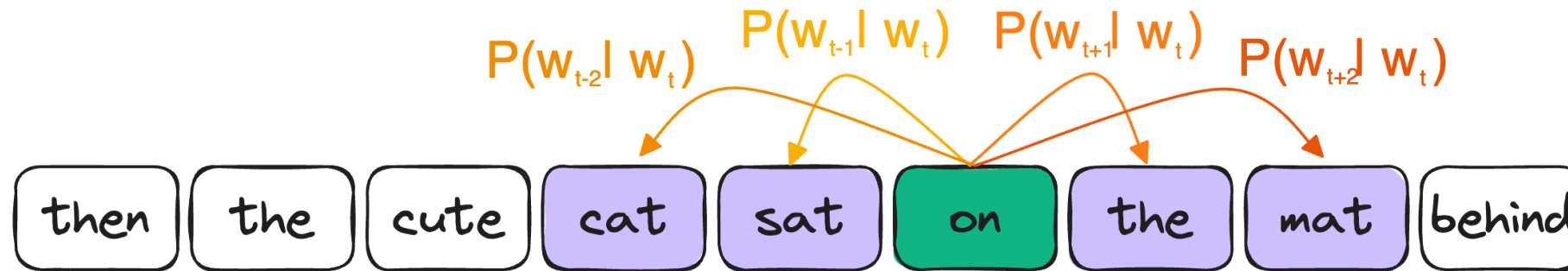
2- Continuous Bag-of-Words (CBOW)

Given surrounding **Context Words** (o), predict the missing **Center Word** (c).



- **Strengths:** Trains several times faster; slightly better accuracy for frequent words.

5.2.2 Skip-gram process



1. Take a huge text corpus;
 2. Go over the text with a sliding window, moving one word at a time.
 3. For the central word, compute probabilities of context words;
 4. Adjust the vectors to increase these probabilities.
- At each step, there is a central word and context words (other words in this window);

5.3 How to compute the probabilities?

5.3.1 Two Vectors Per Word

For every word w in vocabulary $\square$, the model maintains **two separate d -dimensional vectors** (usually, $d \sim 10^2$):

- $\mathbf{v}_w \in \mathbb{R}^d$ when w is acting as a **Center word** (c - central).
- $\mathbf{u}_w \in \mathbb{R}^d$ when w is acting as a **Context word** (o - outside).

$\theta = \{\{\mathbf{v}_w\}_{w \in \square}, \{\mathbf{u}_w\}_{w \in \square}\}$ are the **parameters** of our model.

The **similarity** between center word c and context word o is modeled by their **inner dot product** ($\propto$ cosine similarity):

$$\mathbf{u}_o^T \mathbf{v}_c$$

5.3.2 From similarity to probability

Use the **softmax** function:

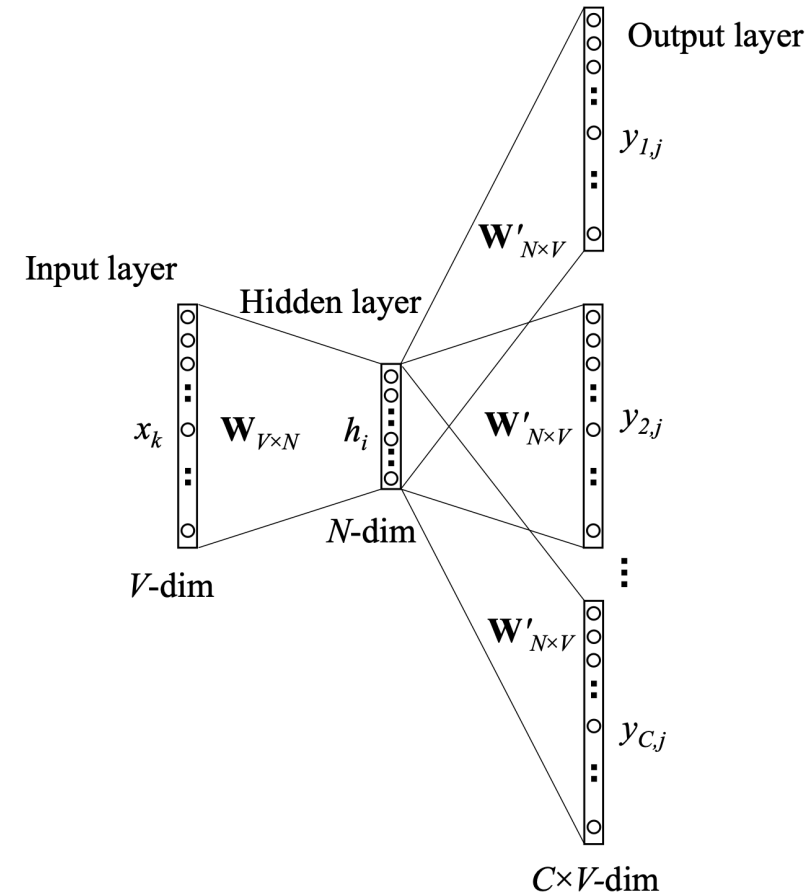
$$P(o \mid c, \theta) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \square} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}$$

- the exponential $\exp(-)$ makes every value positive
- the denominator sum ensures $P \in [0, 1]$, and probabilities sum to 1.

5.3.3 In practice: shallow neural network

- center vectors $\mathbf{v}_w$ rows of $\mathbf{W} \in \mathbb{R}^{|\square| \times d}$
- context vectors $\mathbf{u}_w$ columns of $\mathbf{W}' \in \mathbb{R}^{d \times |\square|}$

1. Input: OHE $\mathbf{x} \in \mathbb{R}^{|\square|}$
2. $\mathbf{h} = \mathbf{x}\mathbf{W} \in \mathbb{R}^d$ (selects the row associated with the OHE)
3. $\mathbf{z} = \mathbf{h}\mathbf{W}' \in \mathbb{R}^{|\square|}$ (logits: dot product from center to every word in vocabulary)
4. Apply softmax (get probabilities)



The skip-gram model. *Extracted from (Rong 2016).*

5.4 Training

5.4.1 Objective

The goal is to **maximise** the **likelihood** of our model θ , given a corpus $\square$ for a specific window size m :

$$\begin{aligned} L(\theta) &= \prod_{c=1}^{|\square|} P(w_{c-m}, \dots, w_{c+m} | w_c) \\ &\stackrel{\text{ind.}}{=} \prod_{c=1}^{|\square|} \prod_{-m \leq j \leq m, j \neq 0} P(w_{c+j} | w_c) \end{aligned}$$

5.4.2 Loss Function

In practice, minimize the **negative log-likelihood**:

$$J(\theta) = \frac{1}{|\square|} \sum_{c=1}^{|\square|} \sum_{-m \leq j \leq m, j \neq 0} J_{c,j}(\theta)$$

with:

$$J_{c,j}(\theta) = -\log\left(\frac{\exp(\mathbf{u}_{c+j}^\top \mathbf{v}_c)}{\sum_{w \in \square} \exp(\mathbf{u}_w^\top \mathbf{v}_c)}\right) = -\mathbf{u}_{c+j}^\top \mathbf{v}_c + \log\left(\sum_{w \in \square} \exp(\mathbf{u}_w^\top \mathbf{v}_c)\right)$$

5.4.3 Negative Sampling (SNGS)



Computational Bottleneck

Before: for each pair of a central word and its context word, we had to update all vectors for context words. This is highly inefficient: for each step, the time needed to make an update is proportional to the vocabulary size (usually, $|\mathcal{V}| > 10^5$).

Solution: **Negative Sampling** ([Mikolov, Sutskever, et al. 2013](#)) → sample only K ‘negative examples’; and use $P_+ = \sigma(x)$, $P_- = \sigma(-x)$, with $\sigma(x) = \frac{1}{1+e^{-x}}$.

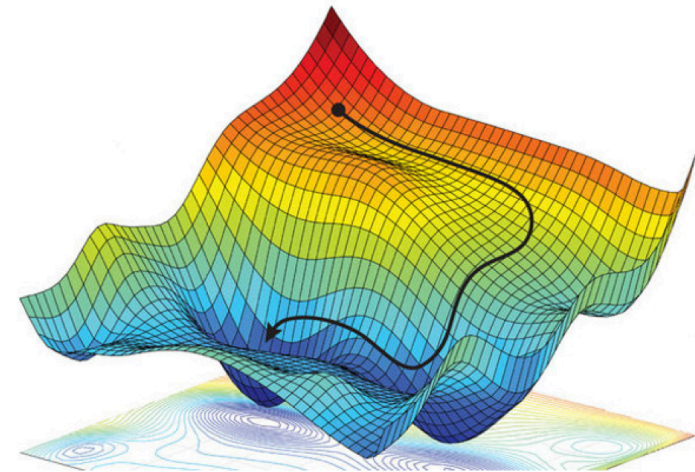
The loss becomes:

$$J_{c,j} = -\log(\sigma(\mathbf{u}_{c+j}^\top \mathbf{v}_c)) + \sum_{w \in \{w_{i1} \cdots w_{iK}\}} \log(\sigma(-\mathbf{u}_w^\top \mathbf{v}_c))$$

5.4.4 Gradient Descent

For word2vec, given learning rate η , at each step, the parameters are updated according to:

- $\mathbf{v}_c^{n+1} = \mathbf{v}_c^n - \eta \frac{\partial J_{c,j}(\theta)}{\partial \mathbf{v}_c}$
- $\mathbf{u}_w^{n+1} = \mathbf{u}_w^n - \eta \frac{\partial J_{c,j}(\theta)}{\partial \mathbf{u}_w} \quad \forall w \in \square$



Gradient Descent Illustration. *Extracted from (Amini et al. 2019).*

6 Navigating Embedding Spaces

6.1 Embedding space structure

6.1.1 Training **word2vec** in practice

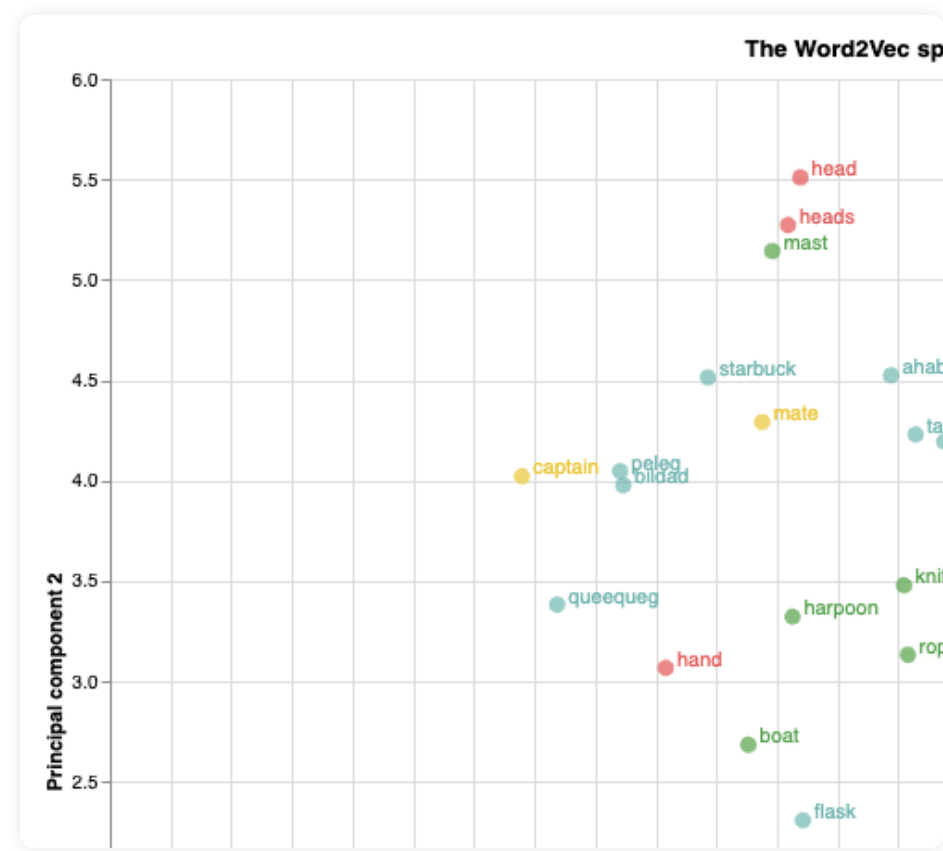
```
1 from gensim.models import Word2Vec
2
3 # Load corpus, split sentences, tokenize,
4 # Define parameters, ...
5 # [ ... ]
6
7 model = Word2Vec( # instantiate & trains model
8     sentences=sentences, # list[list] of tokenized sentences
9     size=embedding_dimension, # size of the embeddings
10    sg=1, # skipgram rather than cbow
11    window=window_size, # size of the window
12    epochs=n_epochs, # number of training passes
13 )
```

6.1.2 Global structures

```
1 from gensim.models import Word2Vec
2 # [ ... ]
3 model = Word2Vec( # instantiate & trains model
4     sentences=sentences, # list[list] of tokenized s
5     size=embedding_dimension, # size of the embeddin
6     sg=1, # skipgram rather than cbow
7     window=window_size, # size of the window
8     epochs=n_epochs, # number of training passes
9 )
10
11 vectors = model.wv[ # stores word vectors
12     words # list of words (in vocabulary)
13 ]
```

- Broad clusters by meaning, language, part-of-speech, etc.
- Global directions capturing abstract semantic attributes can be identified (cf. *analogies*)

t-SNE visualisation of custom Moby Dick `word2vec`:



6.1.3 Local structures

```
1 from gensim.models import Word2Vec
2 # [ ... ]
3 model = Word2Vec( # instantiate & trains model
4     sentences=sentences, # list[list] of tokenized s
5     size=embedding_dimension, # size of the embeddin
6     sg=1, # skipgram rather than cbow
7     window=window_size, # size of the window
8     epochs=n_epochs, # number of training passes
9 )
10
11 vectors = model.wv.most_similar( # cosine similar
12     word, # word of interest (in vocab)
13     topn=n_similar, # number of similar words to ret
14 )
```

- **smaller** context windows (e.g., ± 2) $\Rightarrow$ nearest words are **syntactically** similar words in same taxonomy
- **larger** context windows (e.g., ± 5) $\Rightarrow$ nearest words are related words in same **semantic** field

Example using custom `word2vec` trained on *Moby Dick*:

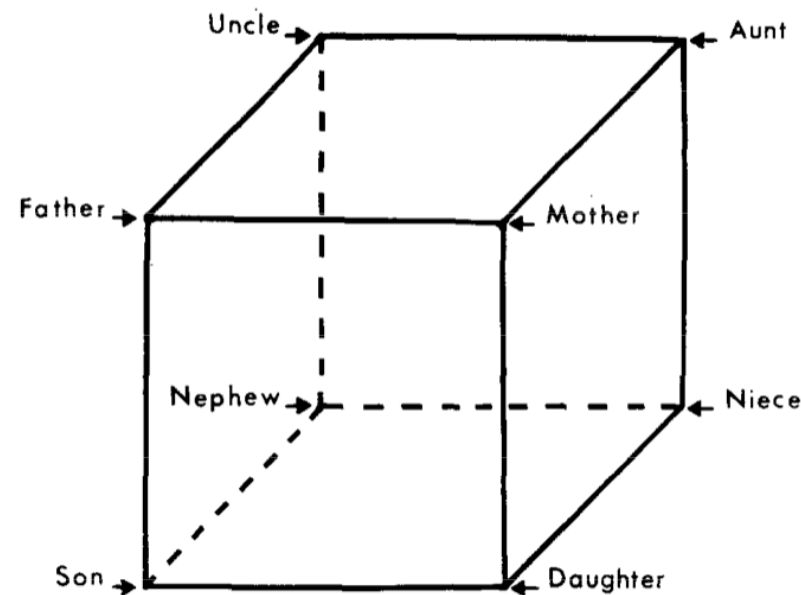
- **ahab's** nearest neighbours:

```
[('vehemently', 0.5342268347740173),
 ('mutter', 0.5329338312149048),
 ('headway', 0.529466986656189),
 ('inaccessible', 0.5006495714187622),
 ('respecting', 0.5005402565002441),
 ('parsee', 0.49932175874710083),
 ('foreboding', 0.4877033829689026),
 ('laughs', 0.48322594165802),
 ('pacing', 0.4829474985599518),
 ('unfaltering', 0.4828788638114929)]
```

6.2 Analogies

6.2.1 The parallelogram model

Parallelogram model of analogical reasoning ([Rumelhart and Abrahamson 1973](#)):



Three-dimensional representation of relations among eight kinship terms. Extracted from ([Rumelhart and Abrahamson 1973](#)).

6.2.2 Analogies & Vector arithmetic

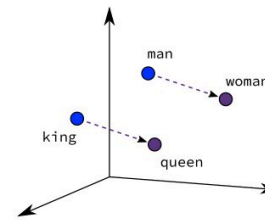
```

1 vectors = model.wv.most_similar( # cosine similarity
2   positive=["B", "C"], # "+" words
3   negative=["A"], # "-" words
4   topn=n_similar, # number of similar words to retrieve
5 )

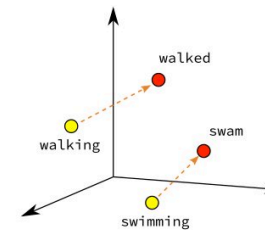
```

Some examples (using `gensim`'s pre-trained 'word2vec-google-news-300'):

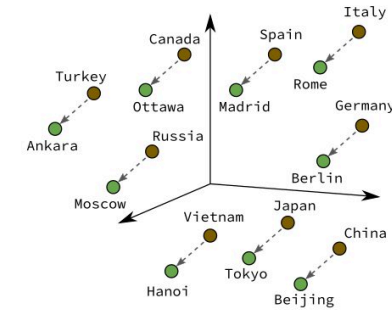
- $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ ($\rightarrow$ top 3: ('queen', 0.71), ('monarch', 0.62), ('princess', 0.59))
- $\text{france} - \text{paris} + \text{berlin} \approx \text{germany}$ ($\rightarrow$ top 3: ('germany', 0.51) ('european', 0.49) ('german', 0.47))



Male-Female



Verb Tense



Country-Capital

Toy illustration of word-vector relationships. [source](#)

- $\text{croque_monsieur} - \text{france} + \text{germany} \approx \text{Wiener_schnitzel}$ ($\rightarrow$ top 3: ('Wiener_schnitzel', 0.57) ('pork_schnitzel', 0.56) ('veal_parmigiana', 0.56))

6.3 Embeddings as a window onto the world

6.3.1 Bias and cultural stereotypes

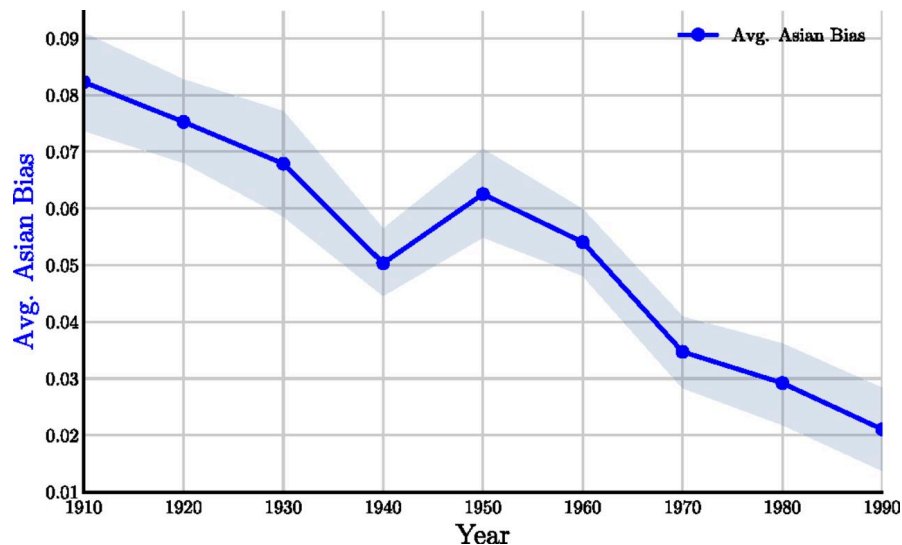
“Man is to computer programmer as woman is to homemaker?”

Bolukbasi et al. (2016)

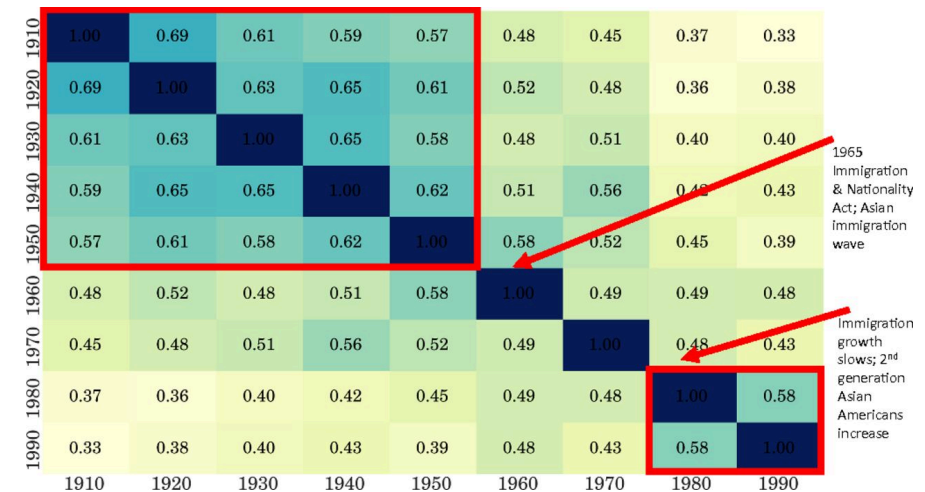
- $\text{computer_programmer} - \text{man} + \text{woman} \approx \text{homemaker}$ (
→ top 3: ('homemaker', 0.56) ('housewife', 0.51) ('graphic_designer', 0.51))
- $\text{doctor} - \text{father} + \text{mother} \approx \text{nurse}$ (→ top 3: ('nurse', 0.71)
('doctors', 0.66) ('gynecologist', 0.65))

6.3.2 Bias and cultural stereotypes, over time

“Word embeddings quantify 100 years of gender and ethnic stereotypes” (Garg et al. 2018)



Asian bias score over time related to dehumanizing adjectives (e.g., *bizarre*, *greedy*, *monstruous*, ...) in COHA data.



Pearson correlation in embedding Asian bias scores for adjectives over time between embeddings for each decade.

6.4 Semantic Shifts

6.4.1 How to study semantic shifts using word2vec?



The Illustrated News. Feb. 16th 1867



Los Angeles Time. May 2nd 1992

→ Today's [Hands-on Exercise: Semantic Shift Analysis](#) through **word2vec** representations, based on ([Pedrazzini and McGillivray 2022](#)) ([Living With Machines project](#)).

6.4.2 Aligning embedding spaces...



Geometry vs. Coordinates

word2vec relies on *stochastic optimisation* to map words into space. The algorithm **optimises a geometry** (relationships between word-vectors), **not a coordinate system**.

In other words, two **word2vec** models trained independently (even on the same corpus) cannot be directly compared!



Orthogonal Procrustes transform

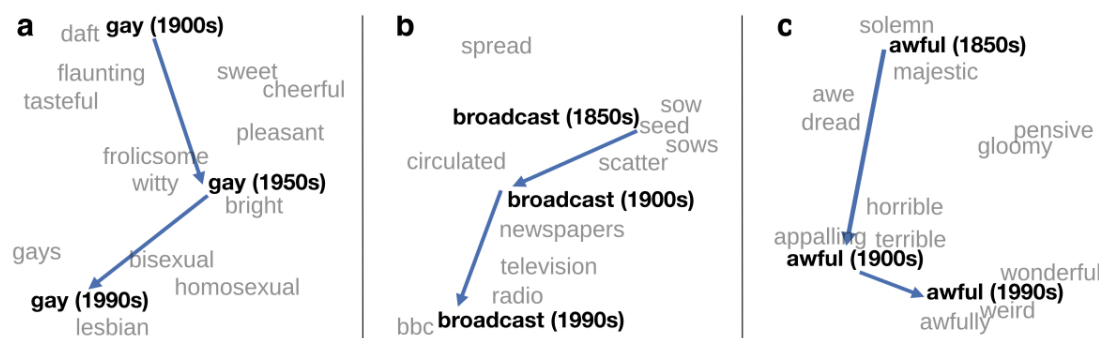
Align embedding spaces through rotation: for $\mathbf{X}, \mathbf{Y} \in \mathbb{R}^{|\square| \times d}$ resp. the target and reference representations, find the orthogonal matrix $\mathbf{W} \in \mathbb{R}^{d \times d}$ that minimizes:

$$\|\mathbf{XW} - \mathbf{Y}\|_F$$

The closed-form solution exists and can be computed efficiently using singular value decomposition (SVD).

6.4.3 Aligning embedding spaces... for diachronic analyses

“*HistWords: Word Embeddings for Historical Text*”
([Hamilton, Leskovec, and Jurafsky 2016](#))



2D visualisation of semantic change in English based on word2vec embeddings trained on “six large historical datasets”. Extracted from ([Hamilton, Leskovec, and Jurafsky 2016](#)).



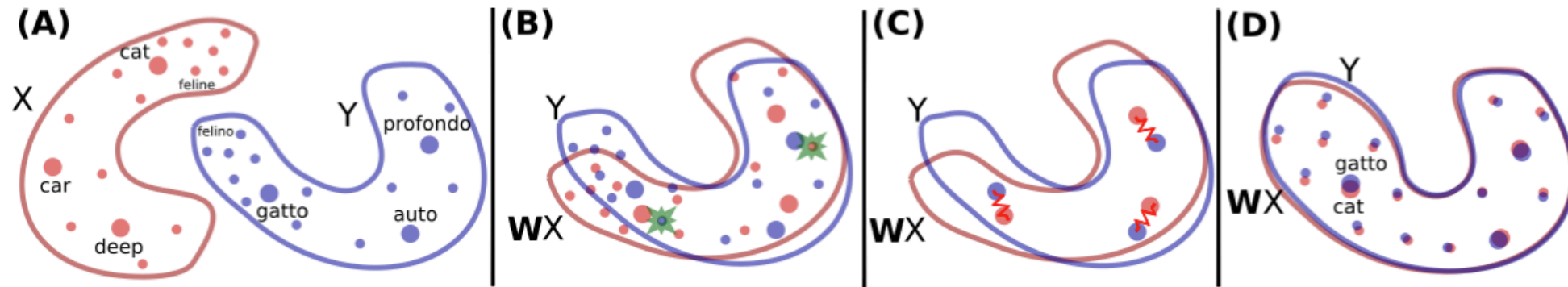
Simpler is better?

Gonen et al. ([2020](#)) propose a simpler approach: focusing on *words' local neighborhoods* (instead of alignment).

They claim that their method is simpler, more interpretable, and more stable.

6.4.4 Aligning embedding spaces... for translation

Aligning embedding spaces from different languages to find correspondences across languages ([Conneau et al. 2018](#)).



Toy illustration of the alignment strategy between embedding spaces of different languages. *Extracted from* ([Conneau et al. 2018](#))

7 Conclusion

- Tokenization: **raw text** → **discrete units** (& Vocabulary mapping)
- Long, sparse representations: OHE & word-word co-occurrence
- Distributed, short, dense representations (learned with **word2vec**)
 - Explorable semantic spaces that encode the knowledge & biases of the training corpus

References

- Amini, Alexander, Ava Soleimany, Sertac Karaman, and Daniela Rus. 2019. “Spatial Uncertainty Sampling for End-to-End Control.” <https://arxiv.org/abs/1805.04829>.
- Bellman, Richard. 1966. “Dynamic Programming.” *Science* 153 (3731): 34–37.
- Bolukbasi, Tolga, Kai-Wei Chang, James Zou, Venkatesh Saligrama, and Adam Kalai. 2016. “Man Is to Computer Programmer as Woman Is to Homemaker? Debiasing Word Embeddings.” In *Proceedings of the 30th International Conference on Neural Information Processing Systems*, 4356–64. NIPS’16. Red Hook, NY, USA: Curran Associates Inc.
- Coenen, Andy, and Adam Pearce. 2019. “Understanding Umap.” *PAIR Page Redirection*. <https://pair-code.github.io/understanding-umap/>.
- Conneau, Alexis, Guillaume Lample, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. 2018. “Word Translation Without Parallel Data.” <https://arxiv.org/abs/1710.04087>.
- Domingos, Pedro. 2012. “A Few Useful Things to Know about Machine Learning.” *Communications of the ACM* 55 (10): 78–87. https://dl.acm.org/doi/abs/10.1145/2347736.2347755?fbclid=IwAR0ribOrR3TwrKh0rOWW6gsvF1CuoXIfcPizvw_ReFN7oOM.J229o5Ar2uAs.